COMPLETE STUDY MATERIAL

A2A Protocol — Complete Notes

Simple Examples • JSON-RPC, gRPC & HTTP+JSON • Core Concepts • Python SDK • End-to-End Flows

PART I — CONCEPT & COMMUNICATION BINDINGS

Understanding A2A with a Simple Example

Let's understand A2A using just one simple example. For now, forget about MCP, JSON-RPC, and gRPC.

Suppose You Have Built Two AI Agents

- **Agent A:** Performs research.
- **Agent B:** Writes articles.

Both agents are built using LangGraph.

Situation 1: Both Agents Are in the Same Python Application

Agent A needs Agent B to write an article.

```
research = research_agent.invoke({
    "topic": "AI"
})

article = writer_agent.invoke({
    "research": research
})
```

That's it! We simply pass the output of Agent A to Agent B.

In this situation, A2A is not required.

Situation 2: Agent B Is Running on Your Friend's Server

Now imagine your friend tells you:

"I have developed a Writer Agent, and it is running on my server. You can use it from your own application."

In this case, you cannot directly call `writer_agent.invoke()` because the Writer Agent is not a local object inside your application.

You have two options.

Option 1 — Using a Normal REST API

```
requests.post(
    "https://friend-server.com/write",
    json={"topic": "AI"}
)
```

That's it! Communication is possible without A2A.

Option 2 — Using the A2A Protocol

Your friend exposes the Writer Agent through an A2A Server.

Your application uses an A2A Client to send a message to the remote Writer Agent.

```
# Simplified conceptual code
a2a_client.send_message(
    "Write an article about AI"
)
```

This approach also works, provided both sides have compatible A2A implementations.

Why Was A2A Introduced?

Imagine that 100 different companies have developed 100 different AI agents.

Each company might have its own API structure, request format, response format, and communication mechanisms.

To integrate all these agents, developers would need to understand and implement different integration methods for each system.

A2A solves this interoperability problem by providing a common set of communication rules that allow compatible AI agents to communicate with one another in a standardized way.

That's the fundamental concept behind A2A!

A2A Protocol Bindings: JSON-RPC, gRPC, and HTTP+JSON

1. JSON-RPC

JSON-RPC = JavaScript Object Notation Remote Procedure Call

In JSON-RPC, Agent A sends a JSON message specifying which method needs to be executed.

```
{
  "jsonrpc": "2.0",
  "method": "SendMessage",
  "params": {
    "message": "Find flights to Dubai"
  },
  "id": 1
}
```

Meaning: "Flight Agent, execute the `SendMessage` operation."

2. gRPC

gRPC = gRPC Remote Procedure Calls

In gRPC, Agent A calls a remote function exposed by Agent B.

```
# Conceptual code
flight_agent.SendMessage(request)
```

The data is typically transmitted in a binary format using Protocol Buffers.

Meaning: "Call a method on the remote agent."

3. HTTP+JSON

HTTP = Hypertext Transfer Protocol

In HTTP+JSON, Agent A sends a request to a standard HTTP endpoint.

```
POST /message:send
```

Along with the HTTP request, Agent A sends a JSON body:

```
{
  "message": "Find flights to Dubai"
}
```

Meaning: "Send an HTTP request to a URL and ask Agent B to perform the operation."

Note: The examples above are simplified illustrations, not complete A2A SDK requests.

4. Comparison of JSON-RPC, gRPC, and HTTP+JSON

Feature	JSON-RPC	gRPC	HTTP+JSON
How is the request made?	Method name inside JSON	Remote function call	HTTP URL + method

Feature	JSON-RPC	gRPC	HTTP+JSON
Data format	JSON	Protocol Buffers (binary)	JSON
Network transport	HTTP(S)	HTTP/2	HTTP(S)
Example	`SendMessage` in JSON	`SendMessage()`	`POST /message:send`

These are the three official standard protocol bindings supported by A2A.

Official Reference: A2A Protocol

5. Where Does A2A Fit Into This Architecture?

A2A = A set of standard communication rules that enable AI agents to communicate with each other.

Protocol Binding = The mechanism used to implement those communication rules over a network.

Important Clarification

Both JSON-RPC and HTTP+JSON can use HTTP and JSON for communication.

The key difference is:

- **JSON-RPC:** The operation or method name is specified inside the JSON message.
- **HTTP+JSON:** The operation is identified using the HTTP method and URL endpoint.

Additionally, if both agents are running inside the same LangGraph application, these protocol bindings are not necessarily required. The agents can communicate directly through LangGraph's internal mechanisms.

6. Final Summary

The relationship can be represented as:

```
Agent A
   |
   v
A2A Protocol
   |
   +---- JSON-RPC
   |
   +---- gRPC
   |
   +---- HTTP+JSON
   |
   v
Agent B
```

In simple terms:

Agent A → A2A (JSON-RPC OR gRPC OR HTTP+JSON) → Agent B

All three bindings serve the same fundamental purpose: enabling communication between independent AI agents using the standardized A2A protocol.

The main difference is how requests and responses are structured, transmitted, and processed.

PART II — IMPORTANT A2A TERMS & CONCEPTS (ATTACHED NOTES)

Important Terms and Concepts in A2A (Agent2Agent) Protocol

In A2A (Agent2Agent), some of the most important terms include **Client, Server, Agent Card, Agent Skill, Message, Task, and Artifact**.

Let's understand all these concepts using a single example.

Travel Agent → A2A → Flight Agent

Suppose a user requests:

"Find flights from Bengaluru to Dubai."

1. Most Important A2A Terms

Term	Simple Meaning	Flight Agent Example
1. A2A Client	The agent or application that sends a request to another agent	Travel Agent
2. A2A Server	The agent or application that receives requests and performs tasks	Flight Agent
3. Agent Card	A digital business card containing information about an agent	Name, URL, skills, security requirements
4. Agent Skill	Describes what an agent is capable of doing	Search Flights, Compare Fares
5. Agent Discovery	The process of finding available agents	Travel Agent discovers the Flight Agent
6. Message	Information exchanged between agents	"Find flights to Dubai"
7. Part	The actual content inside a message	Text, file, or JSON data
8. Task	A trackable unit of work assigned to an agent	Flight search task
9. Task ID	A unique identifier assigned to a specific task	`task-101`
10. Task Status	The current execution state of a task	Working, Completed, Failed
11. Artifact	An output or deliverable generated by an agent during task execution	Structured list of available flights
12. Context ID	An identifier used to group related conversations and tasks	`context-501`

These are important concepts defined in the official A2A specification.

Official Reference: A2A Protocol

---

2. Understanding A2A Through a Simple Diagram

```
         Travel Agent
           (A2A Client)
                |
                |
                v
        A2A Communication
                |
                |
      Discover Agent
                |
                v
         Send Message
                |
                v
   Task: Search Dubai Flights
                |
                v
     Status: WORKING
                |
                v
     Status: COMPLETED
                |
                v
        Flight Agent
         (A2A Server)
                |
                v
      Result / Artifact
                |
                v
        Travel Agent
      Receives the Result
```

Important Difference

Message: An instruction or information sent between agents.

Task: A trackable unit of work that an agent is responsible for completing.

Artifact: The actual output or deliverable produced during task execution.

Important Note: Not every message needs to create a Task. The Flight Agent can also return a direct response in the form of a Message.

---

3. Communication and Advanced A2A Terms

Term	Simple Meaning
13. Agent Interface	Defines the agent's endpoint, protocol binding, and supported protocol version
14. Protocol Binding	Defines how A2A communication is implemented using JSON-RPC, gRPC, or HTTP+JSON
15. Agent Capabilities	Features supported by an agent, such as streaming and push notifications
16. Authentication	Verifies who is allowed to access the agent
17. SendMessage	An operation used by the client to send a message to the server
18. GetTask	An operation used to retrieve the current status or result of an existing task
19. Streaming / SSE	Enables receiving live updates while a task is being processed
20. Polling	Repeatedly checking whether a task has been completed
21. Push Notification	Allows the server to send task updates automatically to a configured webhook
22. Task Lifecycle	Represents the different execution stages of a task, such as Submitted → Working → Completed

Official Reference: A2A Protocol Documentation

---

4. Important Classes in the Python A2A SDK

When working with the Flight Agent / Hotel Agent project, you will encounter the following Python classes.

Class	Purpose
`AgentCard`	Defines the agent's information and metadata
`AgentSkill`	Describes the skills and capabilities offered by an agent
`AgentInterface`	Configures the agent's endpoint and protocol binding
`AgentExecutor`	Acts as a bridge between A2A requests and the actual agent execution logic
`RequestContext`	Provides information about an incoming request
`EventQueue`	Allows the agent to send status updates and result events
`DefaultRequestHandler`	Handles incoming A2A requests
`TaskStore`	Stores and manages tasks and their execution states

These are important implementation concepts used in the official Python A2A SDK.

Note: Not all of these classes represent separate objects defined by the A2A wire protocol. Some are SDK-specific classes that simplify server implementation.

---

5. Complete A2A Communication Flow — All Important Terms Together

Let's understand how all these components work together.

```
Travel Agent (A2A Client)
       |
       | 1. Agent Discovery
       v
Flight Agent's Agent Card
       |
       | 2. Check Agent Skills
       |    "Search Flights"
       |
       | 3. SendMessage
       |    "Find BLR to Dubai flights"
       v
Flight Agent (A2A Server)
       |
       | 4. AgentExecutor
       |    Executes the flight search
       |
       | 5. Task Created
       |    Task ID: 101
       |
       | 6. Task Status
       |    WORKING
       |
       | 7. Task Status
       |    COMPLETED
       |
       v
Artifact (Flight Results)
       |
       v
Travel Agent Receives the Result
```

Step-by-Step Explanation

Step 1 — Agent Discovery

The Travel Agent discovers the Flight Agent through its Agent Card.

Step 2 — Agent Skill Verification

The Travel Agent checks whether the Flight Agent supports the required skill, such as searching for flights.

Step 3 — SendMessage

The Travel Agent sends a request to the Flight Agent.

Example:

"Find flights from Bengaluru to Dubai."

Step 4 — Agent Execution

The Flight Agent receives the request, and the `AgentExecutor` invokes the actual flight-search logic.

Step 5 — Task Creation

The Flight Agent creates a trackable task and assigns a unique Task ID.

Example:

`task-101`

Step 6 — Task Processing

The task moves into the `WORKING` state while the Flight Agent searches for flights.

Step 7 — Task Completion

Once the flight search is finished, the task moves into the `COMPLETED` state.

Step 8 — Artifact Generation

The Flight Agent produces an artifact containing the flight search results.

Example:

```
{
  "flights": [
    {
      "airline": "Emirates",
      "route": "BLR-DXB",
      "price": 28000
    },
    {
      "airline": "Air India",
      "route": "BLR-DXB",
      "price": 24500
    }
  ]
}
```

Step 9 — Result Delivery

The Travel Agent receives the result and presents the available flight options to the user.

Important: This is an example of a task-based communication flow. Simple A2A interactions may return a Message directly without creating a Task.

---

6. Recommended Learning Order

To understand A2A effectively, start by learning these seven core terms.

Order	Term
1	A2A Client
2	A2A Server
3	Agent Card
4	Agent Skill
5	Message
6	Task
7	Artifact

Once these seven concepts are clear, it becomes much easier to understand advanced components such as:

- `AgentExecutor`
- `RequestHandler`
- `EventQueue`
- JSON-RPC
- gRPC
- HTTP+JSON

Final Summary

A2A Client discovers an **A2A Server** through its **Agent Card**, checks the available **Agent Skills**, and sends a **Message** requesting some work.

The server processes the request, optionally creates a **Task**, manages its execution status, and returns the result as a **Message** or one or more **Artifacts**.

In simple terms:

A2A Client → Agent Discovery → Agent Card → Agent Skills → SendMessage → A2A Server → Task Execution → Artifact → A2A Client

This is the fundamental communication flow of the Agent2Agent protocol.

Official Reference: A2A Core Concepts

This PDF preserves the user-provided notes, code examples, diagrams and tables. Code labeled as conceptual in the source remains conceptual.